翁恺课 · 第 8 主题

W8 指针与字符串 · 练习

对齐翁恺《程序设计入门——C语言》第 8 讲 · 参考答案含「思路 + 常见错法」· 请先独立完成再看答案

填空 W8-1

① `int x = 5; int *p = &x;` 中 `&x` 是____，`*p` 是____；执行 `*p = 10` 后 x = ____。

填空 W8-2

② 数组名 `a` 代表____（首地址 / 第一个元素）；`int *p = a;` 后 `*p` 是 ____，`p++` 让 p 指向 ____。

填空 W8-3

③ 字符串 `"abc"` 实际占 ____ 个字符（含结束符）；靠 ____ 知道在哪结束；`char s[3] = "abc";` 会 ____（能/不能）安全使用。

改错 W8-4

④ 下面两段代码各有 1 处致命错误（第 1 段：指针未指向就解引用；第 2 段：字符数组没给 '\0' 留位置）。先自己找，再对照答案的 gcc 诊断。

```
/* debug_wk8_401.c - 两处坑：野指针 + 字符串越界 */
#include <stdio.h>

int main(void)
{
    int *p;         /* 坑1：没指向任何地方 */
    *p = 10;        /* 写到一个野地址 */

    char s[5] = "hello";   /* 坑2：hello 要 6 格（含 '\0'） */
    printf("%s\n", s);
    printf("%d\n", *p);
    return 0;
}
```

编程 W8-5

⑤ 用指针写 `void swap(int *a, int *b)`，main 里换 m=7、n=12。示例应输出 `换前：m=7 n=12 / 换后：m=12 n=7`。

编程 W8-6

⑥ 用指针遍历数组 `{3,5,8,2,9}` 求和。示例应输出 `总和 = 27`。

专业场景 **W8-7**

⑦ 写 `int count_char(char s[], char c)` 统计字符串 `"banana"` 里字符 `'a'` 出现几次。示例应输出 `'a'` 出现 3 次。

概念 **W8-8**

⑧ 结合讲义图 W8-2：为什么 W6 的 `swap(x, y)` 换不动、W8 的 `swap(&x, &y)` 换得动？"传地址给函数"到底发生了什么？为什么字符串必须用 '\0' 结尾？

参考答案

① `&x` 是 x 的**地址**（门牌号）；`*p` 是"顺着 p 找到的变量"即 x（解引用）；`*p = 10` 后 x = **10**。

思路：`int *p = &x` 声明并初始化指针；`*p` 是目标变量，`*p = 10` 等价 x = 10。常见错法：把 `int *p = &x` 写成 `int *p = x`（把值当地址存，编译可能给类型警告且解引用是野指针）。

② 数组名 a 代表**首地址**；`int *p = a` 后 `*p` 是 a[0]；`p++` 让 p 指向 a[1]（每次往后挪一个 int）。

思路：数组名就是首地址（常量，不可 `a++`），但指针 p 可移动遍历。常见错法：把 `*p` 和 `p` 混（`*p` 是值、p 是地址）；或以为 `p++` 加 1 字节（其实加一个元素大小）。

③ 占 **4** 个字符（3 个 + 1 个 '\0'）；靠 **'\0'** 知道在哪结束；`char s[3] = "abc";` 会**不能**安全使用（没给 '\0' 留格子，越界）。

思路：字符串长度 = 字符数 + 1（含 '\0'）。常见错法：数组大小刚好等于字符数，把 '\0' 挤出去，`%s` 会"读到别人家"。gcc 会给 `[-Wunterminated-string-initialization]`（见④）。

④ 两处错误：

1. **野指针**：`int *p;` 没初始化就 `*p = 10;`，写到野地址（可能崩溃/踩坏内存）。修：先 `p = &x;` 指向某处再解引用。
2. **字符串越界**：`"hello"` 要 6 格（含 '\0'），`char s[5]` 只给 5 格，'\0' 无处安放。修：`char s[6] = "hello";`（或直接 `char s[] = "hello";` 让编译器数）。

gcc 诊断口径（-Wall -Wextra 实测原文）：

```
wk8_bug_ptr.c:7:8: warning: 'p' is used uninitialized [-Wuninitialized]
    7 |     *p = 10;
      |     ~~~^~~~
wk8_bug_ptr.c:6:10: note: 'p' was declared here

wk8_bug_str.c:6:17: warning: initializer-string for array of 'char' truncates
```

```
  NUL terminator but destination lacks 'nonstring' attribute (6 chars into 5
  available) [-Wunterminated-string-initialization]
```

诊断口径解读：野指针给 [-Wuninitialized]（这是**警告**不是语法错误，编译能过但运行可能崩）；字符串忘留 '\0' 给 [-Wunterminated-string-initialization]。两处都别指望编译期拦，一个要看运行时、一个要靠"数清楚格子"。

⑤ 参考代码（wk8_p5.c，gcc 实跑 0 警告）：

```c
#include <stdio.h>

void swap(int *a, int *b);

int main(void)
{
    int m = 7, n = 12;
    printf("换前：m=%d n=%d\n", m, n);
    swap(&m, &n);
    printf("换后：m=%d n=%d\n", m, n);
    return 0;
}

void swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

实测输出：

```
换前：m=7 n=12
换后：m=12 n=7
```

思路：传 &m、&n 地址，函数内 *a、*b 改的就是原变量。常见错法：仍写 swap(m, n)（传值，换不动）；或交换时漏 tmp = *a 直接覆盖。

⑥ 参考代码（wk8_p6.c，gcc 实跑 0 警告）：

```c
#include <stdio.h>

int main(void)
{
    int a[5] = {3, 5, 8, 2, 9};
    int *p, sum = 0;
    for (p = a; p < a + 5; p++) {
        sum += *p;
```

```
    }
    printf("总和 = %d\n", sum);
    return 0;
}
```

实测输出：

```
总和 = 27
```

思路：p 从首地址 a 走到 `a + 5`（越界即停），`*p` 累加。常见错法：循环条件写 `p <= a + 5`（多算一个越界元素）；或 `sum += p`（把地址当值加）。

⑦ 参考代码（wk8_p7.c，gcc 实跑 0 警告）：

```
#include <stdio.h>

int count_char(char s[], char c);

int main(void)
{
    char s[] = "banana";
    printf("'a' 出现 %d 次\n", count_char(s, 'a'));
    return 0;
}

int count_char(char s[], char c)
{
    int n = 0, i;
    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] == c) n++;
    }
    return n;
}
```

实测输出：

```
'a' 出现 3 次
```

思路：循环条件用 `s[i] != '\0'` 收尾（靠结束符停），比对每个字符计数。常见错法：循环写成 `i < 6`（把长度写死，字符串变了就错）；或把 `s[i] == c` 写成 `s[i] = c`（赋值，恒真）。

⑧ W6 的 `swap(x, y)` 传的是**值**，函数拿到两份**副本**，改的是副本、外面不变。W8 的 `swap(&x, &y)` 传的是**地址**，函数内 `*a`、`*b` 通过门牌号改的是**原变量本身**，所以换得动。**"传地址"= 把门牌号给函数，函数能进原房间改东西。**字符串必须 '\0' 结尾，是因为所有字符串函数（%s、strlen、strcpy…）都**靠 '\0' 判断在哪结束**——没有它，它们会继续读到相邻内存（越界），长度、输出、复制全错。

> 思路：一句话记——"传值换副本、传址改原件"；"'\0' 是字符串的句号"。常见错法：以为字符串长度算上 '\0' 后还能再多塞；或把数组名当可变指针 a++（数组名是常量地址，不可自增，得用 p）。

本练习册为自写原创变式题（对齐翁恺《程序设计入门——C语言》第 8 讲），仅作课堂伴读；改错题与编程题参考代码均用 gcc -Wall -Wextra 实测（MinGW-W64 gcc 16.1.0），输出为真实运行结果（含 gcc 警告原文）。原视频/教材版权归原作者所有。